

The C Standard Library: `math.h`, `stdio.h`, and `string.h`

Prepared by: Khaled Elwan

Objective

The goal of this report is to explain, in simple terms, three of the most commonly used headers from the C Standard Library: **`math.h`**, **`stdio.h`**, and **`string.h`**. For each header, this report explains what it is used for, lists its most important functions with easy-to-understand descriptions, walks through a working code example step by step, and explains how the header behaves differently on an embedded microcontroller compared to a normal PC program.

Introduction

The C language itself is a small language — it does not include built-in support for things like printing text, comparing strings, or calculating a square root. Instead, C relies on a collection of ready-made header files called the **Standard Library**. Each header groups together functions that solve a related problem, so a programmer does not need to rewrite the same logic from scratch every time.

This report focuses on three of the most frequently used headers:

- **`math.h`** — for mathematical calculations (square roots, powers, trigonometry, etc.)
- **`stdio.h`** — for input and output operations (printing and reading data)
- **`string.h`** — for working with text (strings) and raw memory blocks

1. math.h — The Mathematics Library

1.1 What is it?

The **math.h** header contains functions for performing mathematical operations that go beyond simple addition, subtraction, multiplication, and division. Examples include square roots, powers, logarithms, and trigonometric functions such as sine and cosine. Almost all of these functions work with the **double** data type, because mathematical results are often not whole numbers.

1.2 Most Important Functions

Function	What it does	Example
sqrt(x)	Calculates the square root of x	sqrt(16.0) = 4.0
pow(x, y)	Raises x to the power of y (x^y)	pow(2.0, 3.0) = 8.0
fabs(x)	Returns the absolute (positive) value	fabs(-5.3) = 5.3
ceil(x)	Rounds x UP to the nearest whole number	ceil(4.2) = 5.0
floor(x)	Rounds x DOWN to the nearest whole number	floor(4.8) = 4.0
round(x)	Rounds x to the nearest whole number	round(4.5) = 5.0
sin(x), cos(x), tan(x)	Trigonometric functions (input in radians)	sin(0) = 0.0
log(x)	Natural logarithm (base e)	log(2.718) ~ 1.0
log10(x)	Logarithm with base 10	log10(100.0) = 2.0
exp(x)	Calculates e raised to the power x	exp(1.0) ~ 2.718
fmod(x, y)	Remainder of dividing x by y (for decimals)	fmod(5.3, 2.0) = 1.3

1.3 Example Code

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    double result = sqrt(25.0) + pow(2.0, 4.0);
    printf("Result = %.2f\n", result); // Result = 21.00
    return 0;
}
```

Step-by-step explanation: first, **sqrt(25.0)** calculates the square root of 25, which is 5.0. Then, **pow(2.0, 4.0)** calculates 2 raised to the power 4, which is 16.0. Finally, the two results are added together (5.0 + 16.0 = 21.0) and printed with two decimal places using **%.2f**.

1.4 Embedded Systems Note

On some compilers, the math library must be linked explicitly using the flag **-lm** when the program is built. On ARM embedded toolchains (such as **arm-none-eabi-gcc**), this is often handled automatically, but if the linker shows an error such as **undefined reference to 'sqrt'**, adding **-lm** to the linker command usually solves the problem.

2. stdio.h — Standard Input and Output Library

2.1 What is it?

The **stdio.h** header provides functions that let a program communicate with the outside world — for example, printing messages to the screen, reading input typed by a user, or reading and writing files. The most well-known function in this header is **printf()**, which almost every beginner learns first when starting with C.

2.2 Most Important Functions

Function	What it does
printf(format, ...)	Prints formatted text to the screen
scanf(format, ...)	Reads formatted input typed by the user
puts(str)	Prints a simple string and automatically adds a new line
fgets(str, n, stdin)	Safely reads a line of text (preferred over the unsafe gets())
fopen(filename, mode)	Opens a file so it can be read from or written to
fclose(file)	Closes a file that was previously opened
fprintf(file, format, ...)	Prints formatted text into a file instead of the screen
fscanf(file, format, ...)	Reads formatted data from a file
sprintf(buffer, format, ...)	Prints formatted text into a string variable instead of the screen

2.3 Example Code

```
#include <stdio.h>

int main(void)
{
    int age = 21;
    char name[20] = "Khaled";

    printf("Name: %s, Age: %d\n", name, age);

    char buffer[50];
    sprintf(buffer, "Hello, %s!", name);
    printf("%s\n", buffer); // Hello, Khaled!

    return 0;
}
```

Step-by-step explanation: the program first prints the values of the variables **name** and **age** directly to the screen using **printf**. Then, instead of printing directly, **sprintf()** builds a new text message inside the **buffer** variable. Finally, that buffer is printed to the screen with a normal **printf** call.

2.4 Embedded Systems Note

On a normal PC, **printf()** works immediately because the operating system already provides a "standard output" (the console). On a bare-metal microcontroller, such as an STM32 chip with no operating system, there is no console by default, so **printf()** does nothing on its own. To make it work, the program must **retarget** it — meaning a low-level function such as **_write()** is re-implemented so that the characters printf wants to output are instead sent through a UART peripheral. This is commonly done inside a file such as **syscall.c** in an STM32 project.

3. string.h — String and Memory Library

3.1 What is it?

The **string.h** header provides functions for working with C strings. In C, a string is simply an array of characters that ends with a special character called the **null terminator** (written as **'\0'**). This header also includes general functions for working directly with blocks of memory, which are useful even outside of text processing.

3.2 String Functions

Function	What it does	Example
strlen(str)	Returns the length of a string (not counting '\0')	strlen("Hello") = 5
strcpy(dest, src)	Copies one string into another variable	strcpy(dest, "Hi")
strncpy(dest, src, n)	Copies only the first n characters (safer)	-
strcat(dest, src)	Joins (concatenates) two strings together	strcat(a, b)
strcmp(str1, str2)	Compares two strings; returns 0 if they are equal	strcmp("abc", "abc") = 0
strncmp(str1, str2, n)	Compares only the first n characters	-
strchr(str, ch)	Finds the first occurrence of a character	-
strstr(str1, str2)	Finds one string inside another string	-

3.3 Memory Functions

Function	What it does
memcpy(dest, src, n)	Copies n bytes of raw memory from one location to another
memset(ptr, value, n)	Fills n bytes of memory with a chosen value
memcmp(ptr1, ptr2, n)	Compares n bytes of memory between two locations

3.4 Example Code

```
#include <stdio.h>
#include <string.h>

int main(void)
{
    char greeting[50] = "Hello";
    char name[] = "Khaled";

    strcat(greeting, " ");
    strcat(greeting, name);
    strcat(greeting, "!");

    printf("%s\n", greeting);        // Hello, Khaled!
```

```
    printf("Length: %lu\n", strlen(greeting));

    if (strcmp(name, "Khaled") == 0)
        printf("Names match!\n");

    return 0;
}
```

Step-by-step explanation: the program starts with the string **"Hello"** stored in **greeting**. It then uses **strcat()** three times in a row to gradually build up the message, first adding **", "**, then the name **"Khaled"**, then an exclamation mark. After that, **strlen()** calculates how many characters the final message contains, and **strcmp()** checks whether the **name** variable is exactly equal to the text **"Khaled"**.

3.5 Embedded Systems Note

Functions such as **memcpy** and **memset** are used constantly in embedded programming, not only for text. They are commonly used to copy sensor data between buffers (for example, data coming from an ADC or a UART peripheral), or to quickly clear an entire array to zero before using it. These functions are usually highly optimized by the compiler, which makes them faster and more reliable than writing a manual loop.

Summary and Conclusion

This report explained three core headers of the C Standard Library. Each one solves a different type of problem: **math.h** handles calculations, **stdio.h** handles communication with the outside world, and **string.h** handles text and memory manipulation. The table below summarizes the purpose of each header and how it is typically used in embedded systems programming.

Library	Main Purpose	Typical Use in Embedded Systems
math.h	Mathematical calculations	Control-loop / PID calculations, sensor signal processing
stdio.h	Input and output	Sending debug messages over UART (after retargeting printf)
string.h	Text and memory handling	Processing communication buffers and protocol data

Key Takeaways

- The C Standard Library saves time by providing ready-made functions instead of writing everything from scratch.
- math.h functions mostly work with the double data type and may require the -lm linker flag on some toolchains.
- printf() from stdio.h does not work automatically on a bare-metal microcontroller — it must be retargeted to a hardware peripheral such as UART.
- string.h functions work with null-terminated character arrays, while memory functions (memcpy, memset, memcmp) work with raw bytes and are widely used in embedded buffer handling.